

JAVA INTERVIEW PREPARATION

CHAPTER 39

SPRING EXCEPTION HANDLING
AND PROBLEM DETAILS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Spring Exception Handling and Problem Details: Consistent Failures Without Leaking Internals

Senior / Lead Java Developer Reading Chapter

An API error response is part of the public contract. It must help a client decide whether to correct input, authenticate, retry, resolve a conflict, or contact support. It must also preserve the internal cause for operators without exposing stack traces, SQL, credentials, or service topology.

These goals require two different views of the same failure. The client receives a stable, deliberately limited explanation. Operators receive the causal chain, request context, trace, dependency, and deployment information needed to diagnose it. Returning the internal exception is unsafe; discarding it and logging only "request failed" is operationally useless. A well-designed exception boundary translates without destroying evidence.

The HTTP response is also not the exception taxonomy itself. Several internal causes can map to the same public status, and the same infrastructure exception may require different treatment depending on the operation. The mapping belongs at a boundary that understands both application meaning and protocol semantics.

Exception Flow in MVC

```
controller/service throws
-> HandlerExceptionResolver chain
-> local @ExceptionHandler
-> applicable @ControllerAdvice
-> framework resolvers
-> fallback container error handling
```

Filter-chain failures occur outside this controller pipeline unless explicitly delegated. Spring Security has its own authentication and access-denial handlers.

`DispatcherServlet` asks its ordered `HandlerExceptionResolver` instances to resolve an exception. `ExceptionHandlerExceptionResolver` finds local `@ExceptionHandler` methods and applicable advice; other resolvers handle response-status and framework exceptions. If a resolver writes a response or returns a model and view, the exception is considered handled. If none does, it escapes toward the servlet container and Spring Boot's error dispatch may render a fallback response.

This distinction explains why a global advice can handle a controller's `OrderNotFoundException` but not necessarily an authentication failure raised before MVC. It also explains duplicate error bodies when custom filters catch an exception, write JSON, and then rethrow it into another error mechanism. Each layer needs explicit ownership.

HTTP request	
-> servlet/security filters	failure -> filter/security handler
-> DispatcherServlet	
-> binding and controller	failure -> resolver/advice chain
-> async continuation	failure -> async dispatch/resolution
-> response serialization	failure -> converter/fallback path

ProblemDetail

Modern Spring supports RFC-style problem details with fields such as type, title, status, detail, and instance, plus custom properties.

```
@ExceptionHandler(OrderNotFoundException.class)
ResponseBody<ProblemDetail> notFound(
    OrderNotFoundException exception,
    HttpServletRequest request) {
    ProblemDetail problem = ProblemDetail.forStatus(HttpStatus.NOT_FOUND);
    problem.setTitle("Order not found");
    problem.setDetail("The requested order is unavailable");
    problem.setInstance(Uri.create(request.getRequestURI()));
    problem.setProperty("code", "ORDER_NOT_FOUND");
    return ResponseEntity.status(problem.getStatus()).body(problem);
}
```

Use a stable machine-readable `code` or problem type. Human detail text can change or be localized and should not drive client logic.

RFC 9457 defines the standard shape but does not invent the application's problem taxonomy. `type` should identify a class of problem, ideally through a stable URI under organizational control. `instance` identifies this occurrence or request target, not the exception class. Extensions such as `code`, `traceId`, or bounded validation violations are useful when their semantics are documented.

Spring uses the `status` held by `ProblemDetail` to determine the response status and favors `application/problem+json` during content negotiation. Custom properties are flattened into the JSON representation by Spring's Jackson integration. This convenience makes naming discipline important: an extension should not collide with standard members or expose arbitrary exception metadata.

Do not put dynamic values such as order IDs into `type`. That creates a different problem type for every occurrence and prevents documentation, aggregation, and client handling. Dynamic context belongs in `instance`, safe extensions, or `detail`.

Exception Taxonomy

Separate categories by responsibility:

malformed transport	-> invalid JSON, conversion, missing parameter
structural validation	-> field constraints
domain rejection	-> operation violates business rule
resource absence	-> not found or concealed
concurrency conflict	-> version or state conflict
authentication/authorization	-> security handlers
dependency failure	-> timeout, unavailable, invalid downstream response
unexpected defect	-> internal error

Do not map every `IllegalArgumentException` to 400. It can represent an internal programming defect. Prefer explicit domain exception types or result models.

An exception type should communicate something a caller or boundary can act on. A domain conflict such as `OrderAlreadyShipped` is expected and maps cleanly. A `NullPointerException` is an unexpected defect even if it happened while examining client input. Broad mappings turn programming bugs into client blame and suppress alerts.

Not every expected outcome needs an exception. A use case with several ordinary alternatives may return a sealed result that the controller translates exhaustively. Exceptions remain useful for non-local failure propagation, but using them for every branch can make the normal domain flow invisible.

```
sealed interface CancellationResult {
    record Cancelled(OrderView order) implements CancellationResult {}
    record AlreadyShipped(Instant shippedAt) implements CancellationResult {}
    record NotFound() implements CancellationResult {}
}
```

The choice between result and exception is less important than preserving meaning and avoiding a generic catch-all. The boundary must be able to distinguish correction, conflict, retry, concealment, and defect.

Global Advice

```
@RestControllerAdvice
class ApiExceptionHandler extends ResponseEntityExceptionHandler {
    @ExceptionHandler(DomainConflictException.class)
    ProblemDetail conflict(DomainConflictException exception) {
        ProblemDetail detail = ProblemDetail.forStatus(HttpStatus.CONFLICT);
        detail.setTitle("Operation conflicts with current state");
        detail.setProperty("code", exception.code());
        return detail;
    }
}
```

ControllerAdvice can be ordered and scoped by annotations or packages. Multiple broad advice classes can produce ambiguous handling. Keep ownership clear and test the selected resolver.

Extending `ResponseEntityExceptionHandler` supports customization of framework MVC exceptions, but version-specific method signatures can change during upgrades. Compile and contract-test error behavior.

Advice selection is based on exception compatibility and ordering, so two broad handlers such as `@ExceptionHandler(RuntimeException.class)` can compete in surprising ways. Keep a small number of clearly owned advice classes. A domain advice can translate application failures, while a final fallback handles unexpected defects. Package-scoped advice is useful only when separate APIs genuinely own different contracts.

Return values also matter. A handler returning `ProblemDetail` lets Spring derive its protocol response; `ResponseEntity<ProblemDetail>` is useful when headers such as `Retry-After` or `WWW-Authenticate` are part of the result. Avoid returning a custom envelope from some handlers and `ProblemDetail` from others unless the API intentionally supports both media types.

Validation Errors

Return a bounded list of field violations with safe paths and stable codes.

```
{
  "type": "https://api.example/problems/validation",
  "title": "Request validation failed",
  "status": 400,
  "code": "VALIDATION_FAILED",
  "violations": [
    {"field": "lines[0].quantity", "code": "Min"}
  ]
}
```

Do not echo rejected passwords, tokens, or large payload values. Cap violation count to prevent oversized responses. Object-level validation may not map to one field; support a general path.

Malformed JSON and valid JSON that violates constraints are distinct operational signals even if both return 400.

Spring MVC can produce different exception forms for argument validation and method validation, especially across framework versions and controller signatures. Do not couple the public schema to a framework exception's internal structure. Convert field errors, object errors, and parameter violations into one application-owned violation model.

Field paths need stable semantics. Jackson JSON property names may differ from Java member names, nested collections need indexes, and method parameters may not correspond to body fields. Decide whether public paths use JSON Pointer, dotted property notation, or another documented format. Cap the number of violations and add an `additionalViolationsOmitted` indication rather than producing an unbounded error.

Cross-field rules such as "end must follow start" belong as object-level violations. Do not invent a misleading field just because the UI prefers one. The client can map a general pointer to a form-level message.

Security Errors

Authentication failures and access denial generally occur in Spring Security filters. Configure `AuthenticationEntryPoint` and `AccessDeniedHandler` for consistent problem responses.

Do not reveal whether a protected user, order, or tenant exists. A 404 concealment policy can be safer than 403 in some domains. Apply the policy consistently so timing and message differences do not leak information.

Dependency Failures and Retryability

A downstream timeout is not automatically a client timeout. The service may return 503, 504, a domain fallback, or a partial response depending on gateway role and contract.

Include retry hints only when retry is safe. `Retry-After` can guide temporary capacity failures. Never encourage retry for validation errors, deterministic conflicts, or non-idempotent operations without an idempotency mechanism.

Translation should retain the distinction between timeout, connection refusal, bulkhead rejection, rate limit, malformed downstream response, and downstream domain rejection. They may share a public 503 in some contracts, but operations need the original category to decide whether a circuit breaker opened, a deadline was exceeded, or a release broke deserialization.

Retryability is a property of the operation and failure together. A transient connection failure does not make a payment POST safe to repeat. Conversely, a versioned read may be retried safely within its end-to-end deadline. Error bodies should never invite retry merely because an exception class implements a transient marker.

When this service is acting as a gateway, 502 commonly describes an invalid upstream response and 504 a gateway timeout, while 503 describes temporary unavailability of this service. Exact mapping still follows the public contract and deployment topology; applications should not expose internal service names in the explanation.

Logging Once with Context

Do not log the same exception at repository, service, controller, and filter layers. Repeated stack traces create noise and cost.

expected client/domain rejection	-> structured event, often no stack trace
unexpected defect	-> error with stack trace and correlation ID
dependency failure	-> dependency, timeout, attempt, final outcome

Preserve causes when translating:

```
throw new PricingUnavailableException("pricing unavailable", exception);
```

Never return exception class names, stack traces, SQL, internal hostnames, or raw downstream bodies to clients.

"Log once" does not mean only one telemetry event in the entire system. A dependency client can record a timed span and bounded outcome without logging a stack trace, while the request boundary logs the final unexpected failure with the causal chain. The objective is to avoid five copies of the same exception masquerading as five incidents.

Structured fields should include stable error category, route template, request or trace ID, authenticated subject identifier only if policy permits, and release version. Raw URLs can create high cardinality and leak identifiers. Request bodies and headers should be excluded by default or redacted through an allowlist.

Never swallow `InterruptedException` in a generic wrapper. Restore the interrupt flag or propagate it according to the API so executor cancellation and shutdown remain effective. Avoid catching `Error`; conditions such as `OutOfMemoryError` may leave the process unable to construct a rich JSON response safely.

Correlation and Trace Identifiers

Attach a request correlation or trace identifier to logs and optionally the public problem response. The identifier helps support locate server evidence but should not expose a security token or become an unbounded metric tag.

Generate or validate identifiers at an outer filter so security and unmapped-route failures also carry them. Prevent newline or length abuse in client-provided IDs.

Checked Versus Unchecked at Service Boundaries

Exception type is an API design choice, not an HTTP mapping. Domain failures expected by callers may be explicit result types, checked exceptions, or documented unchecked exceptions. Infrastructure failures are often translated at boundaries.

Do not catch `Exception` merely to wrap everything in one business exception. `InterruptedException`, cancellation, and serious Errors require distinct handling.

Async Exception Handling

Exceptions from `CompletableFuture`, MVC async results, and `@Async` methods do not necessarily travel through the original synchronous call stack. Completion stages must propagate exceptional completion to MVC, and void `@Async` methods need an `AsyncUncaughtExceptionHandler` or a return type that exposes failure.

Background failure cannot be returned to an HTTP request that already received 202. Persist operation status and expose it through the operation resource.

A `CompletableFuture` can hold exceptional completion until someone joins, awaits, or attaches a handler. If a controller returns a supported asynchronous value, Spring can resume processing through an async dispatch and the normal exception mechanisms. If application code launches work and discards the future, no MVC advice can recover the failure. The ownership rule is simple: every asynchronous computation needs an owner that observes its terminal state.

Timeouts deserve their own path. The client may disconnect, the servlet async timeout may expire, or a downstream deadline may fire. Cancellation does not automatically stop blocking I/O, and a timed-out

response does not prove the business operation stopped. Idempotency and operation status are required when late completion would otherwise surprise a retrying caller.

Fallback Error Path

Error handling must itself be resilient. Avoid database queries, remote localization calls, or serialization of complex domain graphs while building an error response.

Have a minimal fallback for failures inside advice or message conversion. If the response is already committed, the server may only close the connection; it cannot replace headers and status reliably.

Design the fallback as a smaller dependency graph than the normal response. It should not load a user profile to localize a message, query the same failing database, or serialize a domain entity with lazy relationships. Stable English title, generic detail, status, and trace ID are often enough when richer construction fails.

Serialization can fail after a controller returned successfully, for example because a lazy association was accessed, a getter threw, or the connection broke during a streamed body. At that point the response might already carry 200 and partial bytes. Metrics should capture committed-response failures separately because the status observed inside the application may not reflect what the client received.

Transaction Boundaries and Exception Translation

Exception mapping must not accidentally change transaction semantics. Catching a runtime exception inside a `@Transactional` method and returning a failure result can allow the transaction to commit unless it has been marked rollback-only. Conversely, translating an exception after it crosses the transaction interceptor normally preserves rollback because the interceptor observed the original failure.

```
@Transactional
public OrderView create(CreateOrder command) {
    try {
        return repository.save(map(command));
    } catch (DataIntegrityViolationException ex) {
        // Translate, but preserve the cause and let the failure leave the boundary.
        throw new DuplicateOrderReference(command.reference(), ex);
    }
}
```

Constraint names and database messages are internal. Translate known constraints to stable domain meaning at the persistence or application boundary, then translate that domain failure to HTTP at the web boundary. This two-stage approach prevents controllers from depending on vendor SQL exceptions.

Be careful with rollback-only transactions. If an inner participant marks the shared transaction rollback-only and outer code suppresses the exception, commit can later fail with `UnexpectedRollbackException`. The public response should describe the actual operation failure, while logs retain both the original cause and rollback outcome.

Testing the Error Contract

Test:

- Malformed JSON and type conversion.
- Field and object validation.
- Domain conflict and absence.
- Authentication and authorization failures.
- Dependency timeout and final retry outcome.

- Unexpected exception with no information leakage.
- Correlation ID presence.
- Content type and stable problem fields.
- Async failure and already-committed responses where relevant.

Snapshot tests can detect accidental schema changes but should allow intended dynamic fields such as timestamp and trace ID.

Use several test layers. Unit tests exercise the pure exception-to-problem mapping. `MockMvc` tests verify advice selection, content negotiation, validation paths, status, headers, and JSON. Security tests verify entry points and access-denied handlers because they do not share the MVC route. A small number of full-stack tests prove that filters, advice, serialization, and Boot fallback behavior compose correctly.

Tests should also prove what is absent: no exception class, SQL fragment, bearer token, rejected password, internal host, or raw downstream body. Negative assertions are especially valuable because leakage often appears only after an upgrade changes a default message.

Production Perspective

Measure error code, route template, dependency, and status family with bounded cardinality. A surge in 400 may indicate a client deployment; a surge in 409 may indicate contention; a surge in 500 after deployment suggests a regression.

Track the original internal cause separately from public status. Mapping every database failure to 500 loses deadlock, constraint, connection, and timeout distinctions needed for operations.

Alerting should use service impact rather than exception volume alone. A frequently rejected business operation might be normal, while one serialization defect on a critical endpoint can be severe. Break down errors by stable public code and internal cause, but bound label values. Sample expected failures and retain full unexpected traces according to privacy and cost policy.

A Strong Interview Answer

I design errors as a public contract and an internal diagnostic record. In Spring MVC, controller failures move through the `HandlerExceptionResolver` chain, while security and early filter failures need their own handlers. I define a small exception or result taxonomy, map it centrally to RFC 9457 `ProblemDetail` with stable problem types and codes, and never expose exception messages blindly. Validation, conflicts, dependency failures, and defects remain distinct because their correction and retry behavior differs. Translation preserves causes and transaction rollback behavior. We log the final failure once with bounded context, observe async completion, keep a minimal fallback path, and contract-test both the response schema and the absence of sensitive information.

Interview-Ready Summary

Spring MVC resolves controller exceptions through `HandlerExceptionResolver`, local handlers, and advice. `ProblemDetail` provides a standard representation, but applications still need stable codes, safe details, and an exception taxonomy.

Security-filter failures require security handlers, not only `ControllerAdvice`. Log expected and unexpected failures differently, preserve causes, expose correlation rather than internals, and design retry hints from idempotency and failure semantics. Test the error contract as carefully as success responses.

Revision Notes

- API errors are public contracts, not serialized stack traces.
- MVC uses an exception-resolver chain.
- Filter and security failures may bypass ControllerAdvice.
- ProblemDetail supports standard and custom fields.
- Clients should branch on stable codes or problem types.
- Separate malformed input, validation, domain, security, and dependency failures.
- Do not map every IllegalArgumentException to 400.
- Keep global advice ownership and ordering clear.
- Validation responses must not echo sensitive rejected values.
- Bound the number and size of violations.
- Security errors need AuthenticationEntryPoint and AccessDeniedHandler.
- Conceal protected resource existence when policy requires it.
- Retry hints require a retryable and idempotent operation.
- Log an exception once at the responsible boundary.
- Preserve root causes when translating exceptions.
- Never expose SQL, stack traces, or internal topology.
- Correlation IDs must be safe and bounded.
- Async failures require explicit completion handling.
- Error construction should not depend on fragile remote work.
- Contract-test status, media type, schema, security, and leakage.